

Next 4 classes

Generic

Collections

Lambdas & Streams

Exception handling

Agenda :-

Generic Types

Raw types

Generic Methods

Bounds

Type Erasure

list < >

lat
long

1) list < list < > >

2) pair → list < pair >
└─→ class pair

double long.

double lat.

3

< country, popu >

class Pair {

Object first;

Object second;

}

→ Java 5

↓

Too flexible

pair Coordi = new Pair ();

Coordi.first = 123.41

Coordi.sec = 243.46

→ no compile
type
check

Coordi.first = "India";

String str = _____

Object str = _____

List<Object> list;

list.add(5);

list.add("Aner");

Pair country & pop = new Pair();

Fortunately, some bugs are easier to detect than others. Compile-time bugs, for example, can be detected early on; you can use the compiler's error messages to figure out what the problem is and fix it, right then and there. Runtime bugs, however, can be much more problematic; they don't always surface immediately, and when they do, it may be at a point in the program that is far removed from the actual cause of the problem.

Generics add stability to your code by making more of your bugs detectable at compile time.

Generics



Parameterised
data types



```
fun (int a) {
```

```
    a = a + 5;
```

```
}
```

```
class Pair <V, S> {
```

```
    V first;
```

```
    S second;
```

```
}
```

```
Pair <Integer, Integer> coord =  
    new Pair();
```

```
coord.first = 123;
```

```
coord.sec = 243;
```

```
X coord.first = "Puneet";
```

```
Pair <String, Double> pep = new Pair();
```

```
pep.first = "India";
```

```
pep.second = 123.45;
```

ArrayList < E >

class ArrayList < E > {
 E
 =

class pair < V, S > {

 V first;
 S second;

 V getFirst() {
 return first;
 }

 S getSecond() {
 return second;
 }

 void setFirst (V data) {
 first = data;
 }

}

```
Pair < Integer, Integer> p1 = new Pair < Integer, Integer> ();
```

↳ java 7.

you can omit this,

```
Pair < Integer, Integer> p1 = new Pair <> ();
```

```
p1.setFirst (7);
```

```
int x = p1.getFirst();
```

```
Pair < String, Integer> p2 = new Pair <> ();
```

```
p2.setFirst ("Puneet");
```

```
int x = p2.getFirst(); X
```

class HashMap {

Object key;
Object value;

HashMap map;
new HashMap();

allowing

class HashMap < K, V > {

K key;
V value;

HashMap <Int, Str>
hmap = new HashMap
<>();

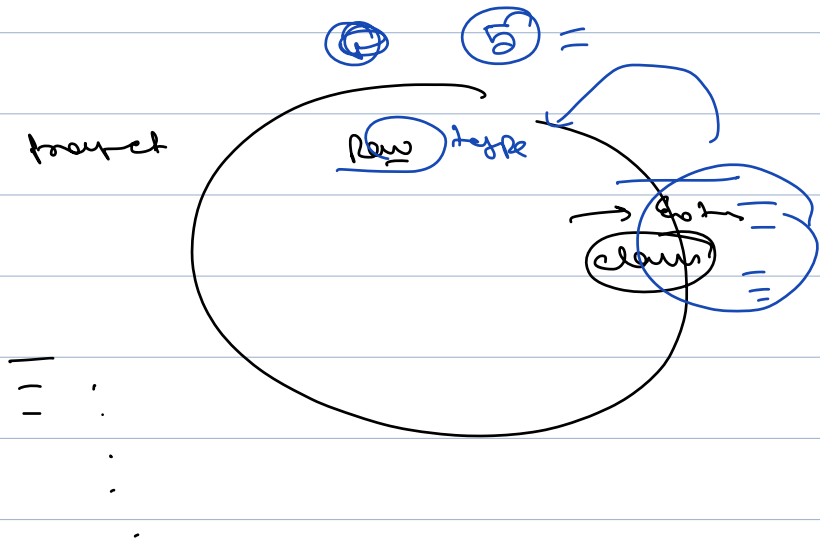
Object
↓ ↘

HashMap < Integer, String >

} → Type Erasure

HashMap < int, - > ; X

10:18pm - 10:28pm Break



from Java docs

Key Points:

- **Avoid Using Raw Types:** Raw types should generally be avoided because they bypass the type safety offered by generics, leading to potential runtime errors.
- **Type Safety:** Using parameterized types ensures type safety and makes your code more robust by catching type mismatches at compile time.
- **Legacy Code:** Raw types are primarily maintained for backward compatibility with older Java code that predates generics.

HashMap < Integer, Integer > tmap = _____
↳ parameterized type

HashMap tmap = _____
↳ raw type

Animal



Dog

Animal a = new Dog();

doSomething (List<Animal> animals) {



List<Dog> dogs;

doSomething (dogs);

X

List<E>



ArrayList<E>

Animal



Dog

ArrayList<Animal>



ArrayList<Dog>

X

List<Dog>



ArrayList<Dog>

doSomething (Animal a) {



doSomething (new Dog());

A
↑
B

hashCode(A)
↑
hashCode(B)

Bounds

↗ upper bound

< T extends Animal > void doSomething(List< T > list)

↓
class / interface

3

Animal
↓ ↓
Dog Cat

Any list from Animal class
and below.

```

public static <T extends Animal> List<T> printAnimalNames(List<T> animals) { no usages new *
//    animals.add(new Animal("Elephant"));
    animals.add(null);
    animals.remove(index: animals.size() - 1);
    for (Animal animal : animals) {
        System.out.println(animal.name);
    }
    return animals;
}

public static void printAnimalNames2(List<? extends Animal> animals) { no usages new *
//    animals.add(new Animal("Elephant"));
//    animals.add(null);
    animals.remove(index: animals.size() - 1);
    for (Animal animal : animals) {
        System.out.println(animal.name);
    }
}

```

→ wildcard ,

<T extends Animal>

upper bound
Animal

<T super Animal> → lower bound

or above .

↑
Animal

Type Erasure

↓

```
class Pair <E, V> {
```

```
    E first;
```

```
    V second;
```

```
    E getFirst() {
```

```
        return first;
    }
```

```
}
```

```
class Pair {
```

```
    Object first;
```

```
    Object second;
```

```
    Object getFirst() {
```

```
        return first;
    }
```

```
}
```

```
Pair <Int, String> p1 = —
```

```
int x = p1.first;
```

```
int x = (Int) p1.first;
```

```
Pair <String, Integer> p2 = —
```

```
int y = p2.first;
```

```
Integer n = 5;
```

```
Pair <int, Boolean> : X
```

```
HashMap<int, int>;
```

Type Erasure → Project